SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05
NAME: Mukesh Raj L
REG NO: 192571036

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

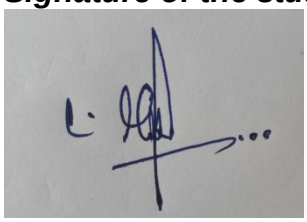
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions	Logical solutions with	Basic solutions with weak	Incorrect or no solution provided
		with strong justification	adequate justification	justification	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1)

1. Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

A single order can contain **multiple products**, so one OrderID can occur in multiple rows.

Example:

OrderID	CustID	CustName	ProdID	ProdName	Qty	Price
O101	C01	Arun	P01	Laptop	1	50000
O101	C01	Arun	P02	Mouse	2	800
O102	C02	Bala	P01	Laptop	1	50000

2. Identify Functional Dependencies

FD 1: Customer ID determines Customer Name

CustID → CustName

A particular CustID identifies one customer.

For example:

C01 → Arun

FD 2: Product ID determines Product Name and Price

ProdID → ProdName, Price

A particular product ID identifies its product name and price.

For example:

P01 → Laptop, 50000

FD 3: Order ID determines Customer ID

OrderID → CustID

An order belongs to one customer.

For example:

O101 → C01

Therefore, through transitivity:

OrderID → CustID
CustID → CustName

So:

OrderID → CustName

is a **transitive dependency**.

FD 4: Order and Product determine Quantity

(OrderID, ProdID) → Qty

The quantity depends on both the order and the product.

For example:

(O101, P01) → 1
(O101, P02) → 2

3. Candidate Key

Because one order can contain multiple products, OrderID alone cannot uniquely identify every row.

Similarly, ProdID alone cannot identify an order line.

Therefore:

(OrderID, ProdID)

is the candidate key.

Closure

(OrderID, ProdID)+

Start with:

{OrderID, ProdID}

Using:

OrderID → CustID

we get:

{OrderID, ProdID, CustID}

Using:

CustID → CustName

we get:

{OrderID, ProdID, CustID, CustName}

Using:

ProdID → ProdName, Price

we get:

{OrderID, ProdID, CustID, CustName, ProdName, Price}

Using:

(OrderID, ProdID) → Qty

we get:

{OrderID, ProdID, CustID, CustName, ProdName, Qty, Price}

Therefore, the closure contains **all attributes**.

Hence:

Candidate Key = (OrderID, ProdID)

4. Problems in the Original Relation

The original relation contains redundancy.

Update Anomaly

If customer C01 changes their name, the customer name must be updated in every order belonging to that customer.

If one row is not updated, inconsistent customer information occurs.

Insertion Anomaly

A new customer cannot easily be stored unless an order is created for that customer.

Similarly, a new product may not be stored independently of an order.

Deletion Anomaly

If the only order containing a particular product is deleted, product information such as ProdName and Price may also be lost.

5. Convert to 2NF

The candidate key is:

(OrderID, ProdID)

But there are **partial dependencies**:

OrderID → CustID
ProdID → ProdName, Price

These attributes depend on only part of the composite key.

Therefore, the relation is not in 2NF.

Decompose it into:

ORDER
ORDER(OrderID, CustID)
PRODUCT
PRODUCT(ProdID, ProdName, Price)

ORDER_ITEM
ORDER_ITEM(OrderID, ProdID, Qty)

6. Convert to 3NF

There is still a transitive dependency:

OrderID → CustID
CustID → CustName

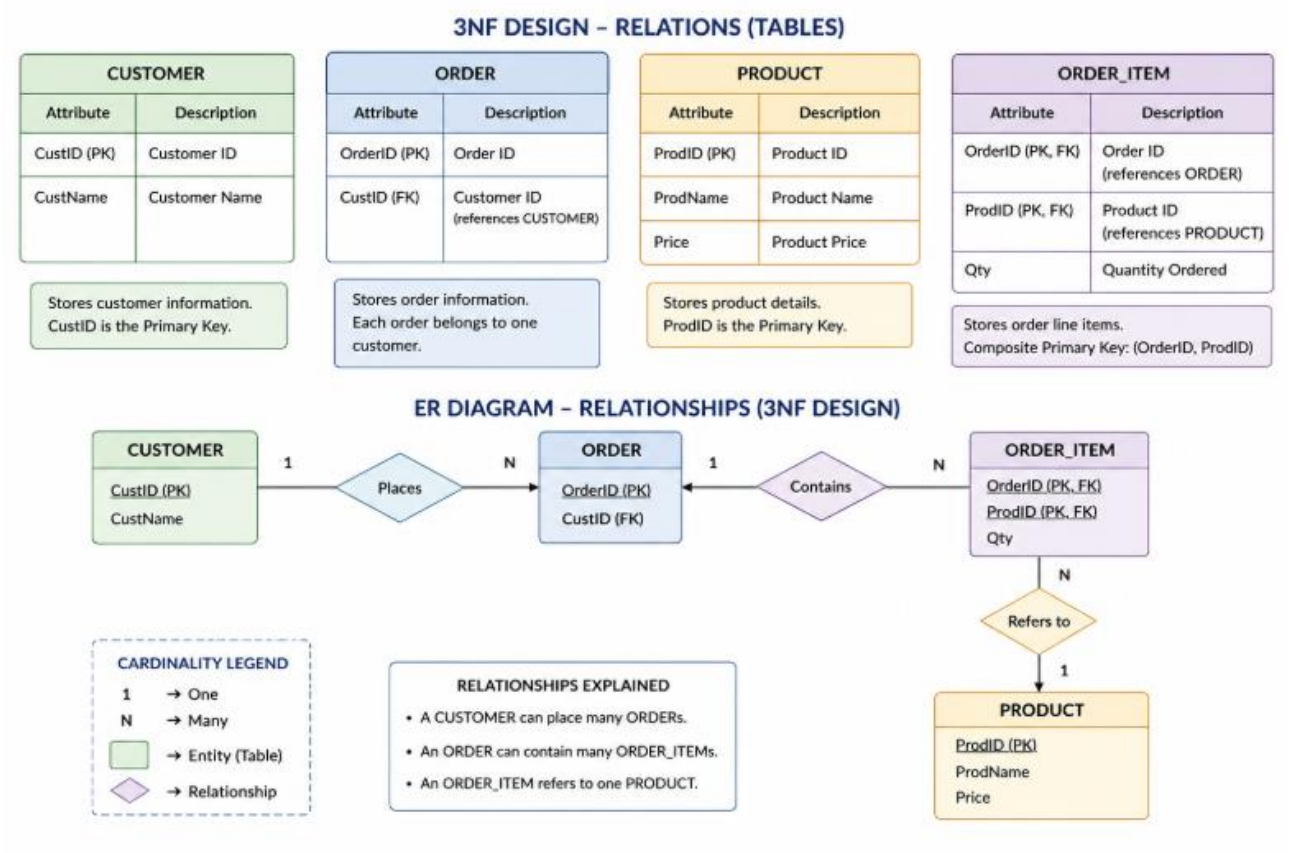
Therefore:

OrderID → CustName

is a transitive dependency.

So customer information must be separated.

CUSTOMER
CUSTOMER(CustID, CustName)
ORDER
ORDER(OrderID, CustID)
PRODUCT
PRODUCT(ProdID, ProdName, Price)
ORDER_ITEM
ORDER_ITEM(OrderID, ProdID, Qty)



2)

1. Functional Dependencies

Based on the meaning of the attributes:

PID → PName, DocID
DocID → DocName, Specialization, WardNo
WardNo → WardName

Therefore, there are also transitive dependencies:

PID → DocID → DocName
PID → DocID → Specialization
PID → DocID → WardNo → WardName

2. Candidate Key

Since PID uniquely identifies a patient:

PID+ = {PID, PName, DocID, DocName, Specialization, WardNo, WardName}

Therefore:

Candidate Key = PID

3. Anomalies

Insertion Anomaly:
If a new doctor or ward is added but no patient is currently assigned to them, their information cannot be stored properly without creating a patient record.

Update Anomaly:
If a doctor's name or specialization changes, the information may have to be updated in multiple patient records. Missing one update can cause inconsistent data.

Deletion Anomaly:
If the only patient assigned to a particular doctor or ward is deleted, information about that doctor or ward may also be lost.

4. BCNF Decomposition

A relation is in **BCNF** if, for every non-trivial functional dependency X → Y, X must be a superkey.

In the original relation:

DocID → DocName, Specialization, WardNo

But DocID is not the key of the complete PATIENT relation.

Also:

WardNo → WardName

But WardNo is not a superkey of the original relation.

Therefore, the relation is **not in BCNF**.

Decompose Doctor Information
DOCTOR(DocID, DocName, Specialization, WardNo)

Here, DocID is the primary key.

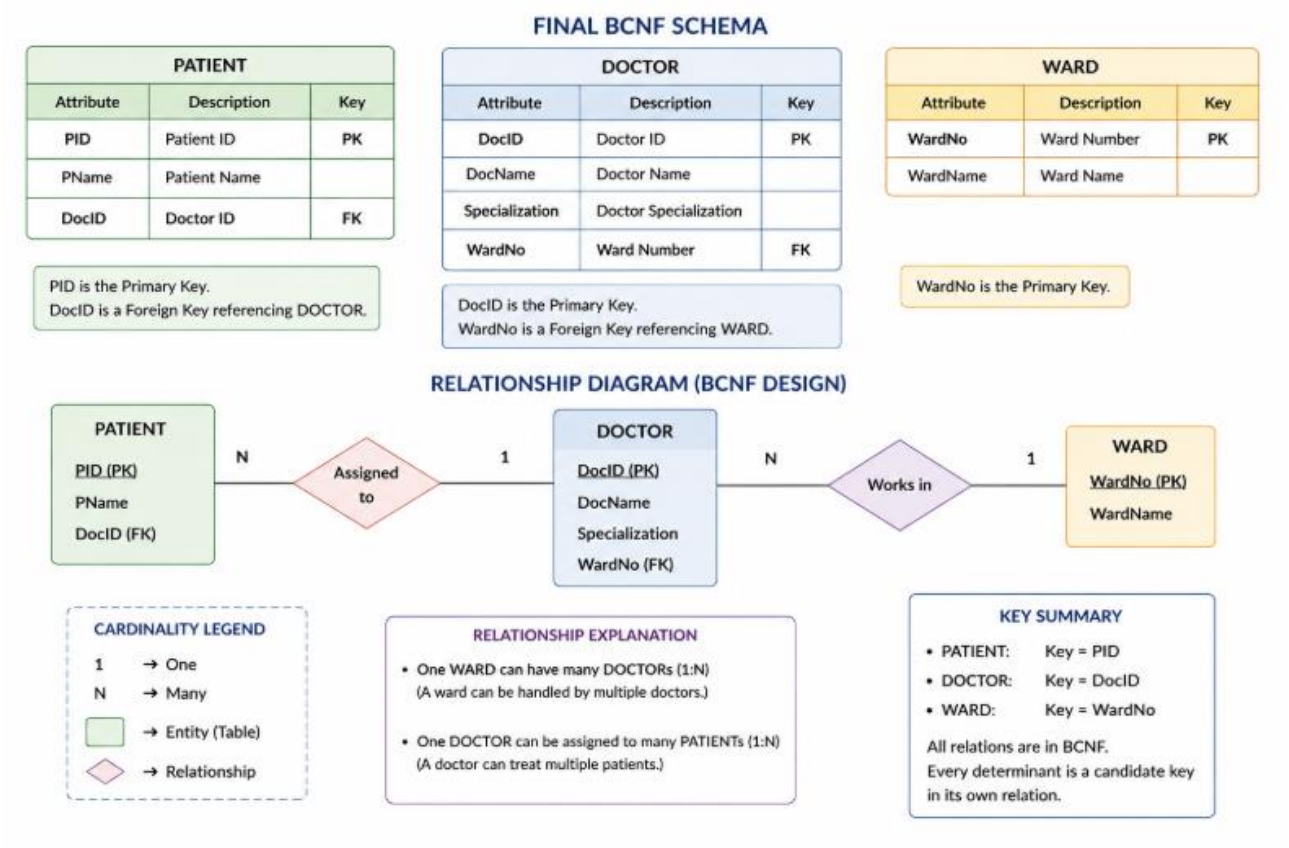
Decompose Ward Information
WARD(WardNo, WardName)

Here, WardNo is the primary key.

Patient Relation
PATIENT(PID, PName, DocID)

Here, PID is the primary key and DocID is a foreign key.

5. Final BCNF Schema



3)

1. Given Relation

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Where:

- SID → Student ID
- SName → Student Name
- CourseID → Course ID
- CourseName → Course Name
- Faculty → Faculty teaching the course
- Grade → Grade obtained by the student

A student can enroll in multiple courses, and a course can have multiple students.

2. Identify Functional Dependencies

FD 1: Student ID determines Student Name

SID → SName

A particular student ID identifies one student name.

FD 2: Course ID determines Course Name

CourseID → CourseName

A particular course ID identifies one course.

FD 3: Course ID determines Faculty

CourseID → Faculty

A particular course is associated with its faculty.

Therefore, these can be written together as:

CourseID → CourseName, Faculty

FD 4: Student and Course determine Grade

(SID, CourseID) → Grade

The grade depends on **which student** took **which course**.

For example, the same student can get different grades in different courses.

3. Complete Functional Dependencies

SID → SName

CourseID → CourseName, Faculty

(SID, CourseID) → Grade

4. Find the Candidate Key

Since one student can take many courses, SID alone cannot uniquely identify an enrollment record.

Similarly, CourseID alone cannot uniquely identify an enrollment record because many students can take the same course.

Therefore, we consider:

(SID, CourseID)
Closure

Start with:

(SID, CourseID)+ = {SID, CourseID}

Using:

SID → SName

we get:

{SID, CourseID, SName}

Using:

CourseID → CourseName, Faculty

we get:

{SID, CourseID, SName, CourseName, Faculty}

Using:

(SID, CourseID) → Grade

we get:

{SID, CourseID, SName, CourseName, Faculty, Grade}

The closure contains **all attributes** of the relation.

Therefore:

Candidate Key = (SID, CourseID)

5. Check for 2NF

A relation is in **2NF** when:

1. It is already in 1NF.
2. There is no partial dependency of a non-key attribute on part of a composite candidate key.

Here, the candidate key is:

(SID, CourseID)

But we have:

SID → SName

SName depends only on SID, which is **part of the composite key**.

This is a **partial dependency**.

We also have:

CourseID → CourseName, Faculty

CourseName and Faculty depend only on CourseID, which is also **part of the composite key**.

Therefore, the original relation is **not in 2NF**.

6. Remove Partial Dependencies

Separate the attributes that depend only on SID and CourseID.

STUDENT Relation
STUDENT(SID, SName)

Primary Key:

SID

COURSE Relation
COURSE(CourseID, CourseName, Faculty)

Primary Key:

CourseID

ENROLLMENT Relation

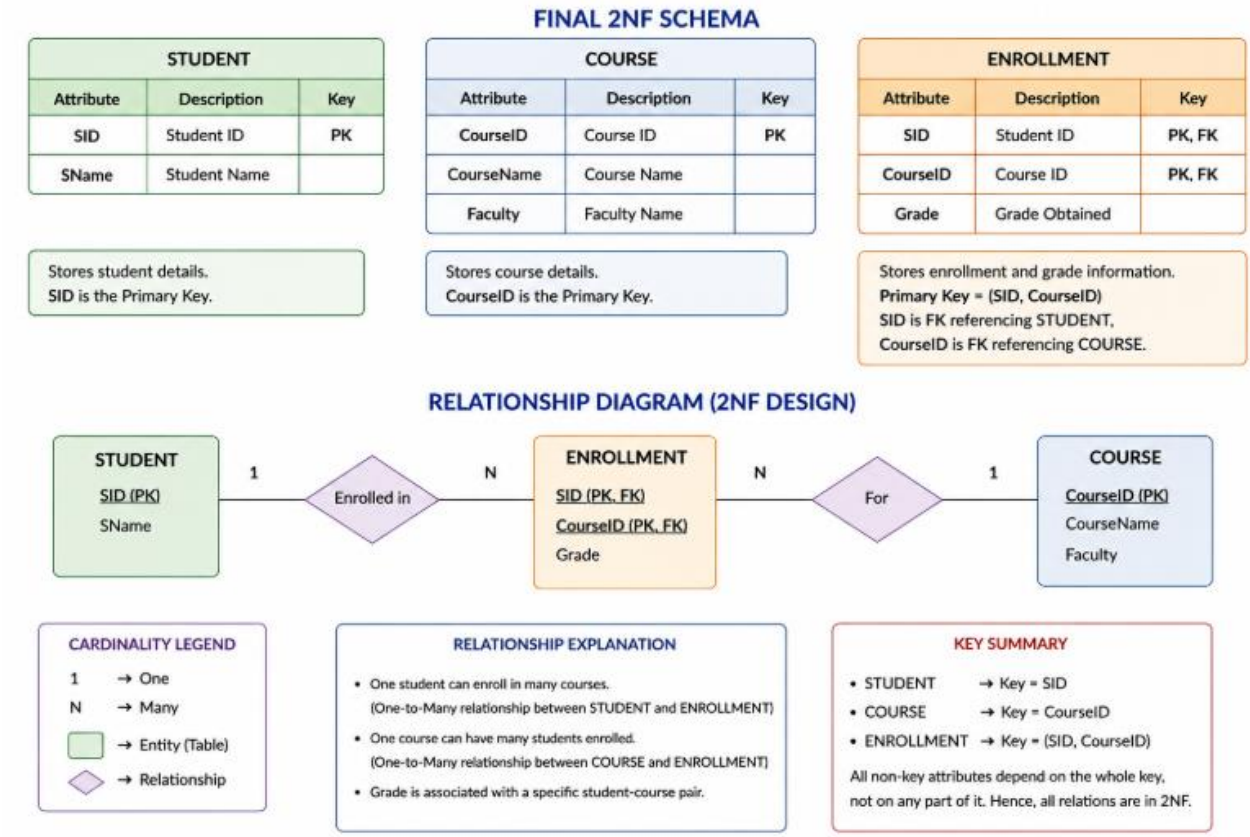
The remaining attribute Grade depends on the complete combination (SID, CourseID).

ENROLLMENT(SID, CourseID, Grade)

Primary Key:

(SID, CourseID)

7. Final 2NF Schema



Meaning:

- One student can have **many enrollment records**.
- One course can have **many enrollment records**.
- ENROLLMENT connects students and courses.
- Grade belongs to the particular student-course combination

4)

1. Given Relation

BOOKING(
FlightNo,
PassID,
PassName,
Seat,
DeptCity,
ArrCity,
Date
)

Where:

- FlightNo – unique flight number
- PassID – passenger ID
- PassName – passenger name
- Seat – seat assigned to the passenger
- DeptCity – departure city
- ArrCity – arrival city
- Date – flight date

2. Functional Dependencies

Dependency 1

PassID → PassName

A passenger ID uniquely identifies the passenger's name.

Dependency 2

FlightNo → DeptCity, ArrCity

A flight number determines its departure and arrival cities.

Dependency 3

For a particular flight on a particular date, the passenger gets a particular seat:

(FlightNo, Date, PassID) → Seat

Thus, the complete booking can be identified by:

(FlightNo, Date, PassID)

3. Candidate Key

Consider:

(FlightNo, Date, PassID)+

Initially:

{FlightNo, Date, PassID}

Using:

PassID → PassName

we get:

{FlightNo, Date, PassID, PassName}

Using:

FlightNo → DeptCity, ArrCity

we get:

{FlightNo, Date, PassID, PassName, DeptCity, ArrCity}

Using:

(FlightNo, Date, PassID) → Seat

we get:

{FlightNo, Date, PassID, PassName, Seat, DeptCity, ArrCity}

This contains all attributes.

Therefore:

Candidate Key = (FlightNo, Date, PassID)

4. Check 1NF

The relation contains atomic values.

There are no repeating groups or multi-valued attributes.

Therefore:

BOOKING

is in **1NF**.

5. Check 2NF

The candidate key is:

(FlightNo, Date, PassID)

It is a composite key.

However:

PassID → PassName

PassName depends only on PassID, which is only part of the composite key.

Also:

FlightNo → DeptCity, ArrCity

DeptCity and ArrCity depend only on FlightNo, which is also only part of the composite key.

Therefore, there are **partial dependencies**, so the original relation is not in 2NF.

Decompose:
PASSENGER(PassID, PassName)
FLIGHT(FlightNo, DeptCity, ArrCity)
BOOKING(FlightNo, Date, PassID, Seat)

6. Check 3NF

PASSENGER
PassID → PassName

PassID is the key.

So it is in 3NF.

FLIGHT
FlightNo → DeptCity, ArrCity

FlightNo is the key.

So it is in 3NF.

BOOKING
(FlightNo, Date, PassID) → Seat

The complete composite key determines Seat.

There is no transitive dependency.

Therefore, the relations satisfy **3NF**.

7. Check BCNF

A relation is in **BCNF** if, for every non-trivial functional dependency:

X → Y

X must be a **superkey**.

PASSENGER
PassID → PassName

PassID is the candidate key.

Therefore, it satisfies BCNF.

FLIGHT
FlightNo → DeptCity, ArrCity

FlightNo is the candidate key.

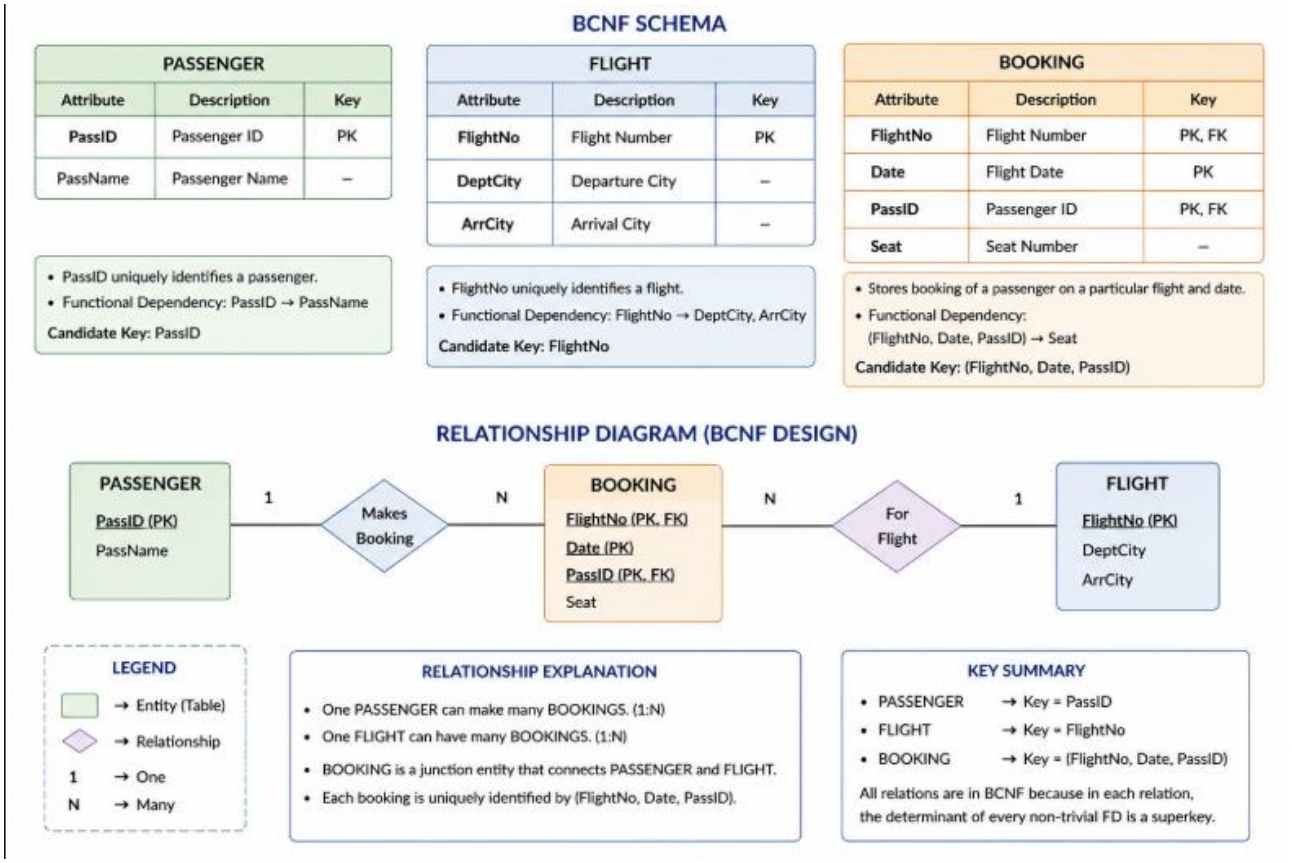
Therefore, it satisfies BCNF.

BOOKING
(FlightNo, Date, PassID) → Seat

(FlightNo, Date, PassID) is the candidate key.

Therefore, it satisfies BCNF.

Hence, all three relations are in **BCNF**.



5)

1. Given HR Relation

Consider the following HR relation:

```
EMPLOYEE(
   EmpID,
   EmpName,
   DeptID,
   DeptName,
   ManagerID,
   ManagerName,
   Salary
)
```

Where:

- EmpID – Employee ID
- EmpName – Employee name
- DeptID – Department ID
- DeptName – Department name
- ManagerID – Manager ID
- ManagerName – Manager name
- Salary – Employee salary

2. Functional Dependencies

Dependency 1
EmpID → EmpName, DeptID, Salary

An employee ID uniquely identifies the employee's name, department, and salary.

Dependency 2
DeptID → DeptName, ManagerID

A department ID identifies the department name and its manager.

Dependency 3
ManagerID → ManagerName

A manager ID uniquely identifies the manager's name.

Therefore, the main functional dependencies are:

EmpID → EmpName, DeptID, Salary

DeptID → DeptName, ManagerID

ManagerID → ManagerName

3. Identify Transitive Dependencies

A transitive dependency occurs when:

A → B
B → C

therefore:

A → C

In our HR relation:

EmpID → DeptID
DeptID → DeptName

Therefore:

EmpID → DeptName

This is a transitive dependency.

Similarly:

EmpID → DeptID
DeptID → ManagerID
ManagerID → ManagerName

Therefore:

EmpID → ManagerName

is also a transitive dependency.

So the transitive dependencies are:

EmpID → DeptName
EmpID → ManagerID
EmpID → ManagerName

4. Candidate Key

Consider the closure of EmpID:

EmpID+

Initially:

EmpID+ = {EmpID}

Using:

EmpID → EmpName, DeptID, Salary

we get:

EmpID+ = {EmpID, EmpName, DeptID, Salary}

Using:

DeptID → DeptName, ManagerID

we get:

EmpID+ =
{EmpID, EmpName, DeptID, DeptName, ManagerID, Salary}

Using:

ManagerID → ManagerName

we get:

EmpID+ =
{EmpID, EmpName, DeptID, DeptName,
ManagerID, ManagerName, Salary}

Since the closure contains all attributes:

Candidate Key = EmpID

5. Normalize to 3NF

The original relation contains transitive dependencies.

Therefore, separate the department and manager information.

EMPLOYEE

EMPLOYEE(
  EmpID,
  EmpName,
  DeptID,
  Salary
)

Primary Key:

EmpID

DEPARTMENT

DEPARTMENT(
  DeptID,
  DeptName,
  ManagerID
)

Primary Key:

DeptID

MANAGER

MANAGER(
  ManagerID,
  ManagerName
)

Primary Key:

ManagerID

6. Foreign Keys

In the EMPLOYEE relation:

DeptID → DEPARTMENT(DeptID)

In the DEPARTMENT relation:

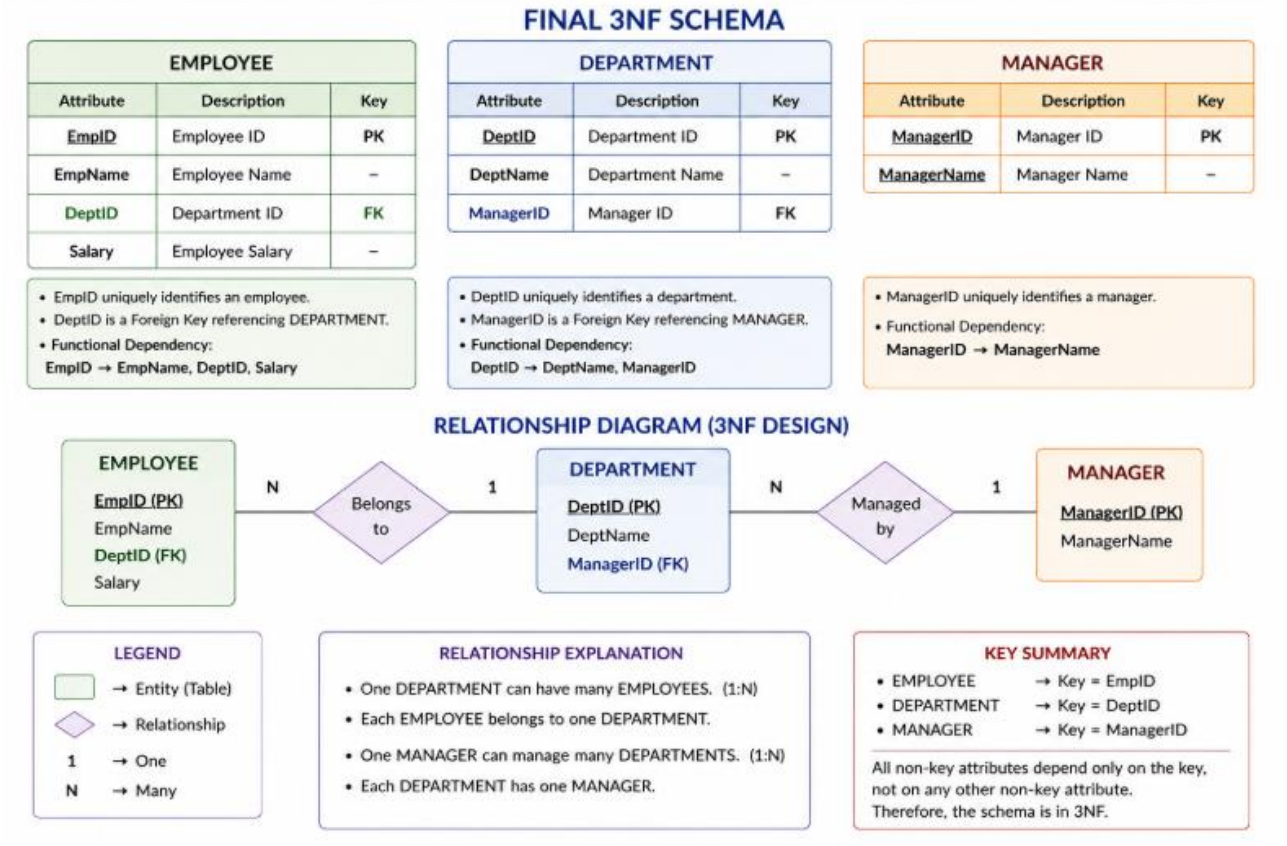
ManagerID → MANAGER(ManagerID)

Therefore:

EMPLOYEE.DeptID
↓
DEPARTMENT.DeptID

DEPARTMENT.ManagerID
↓
MANAGER.ManagerID

7. Final 3NF Schema



8. Why the Decomposition is in 3NF

EMPLOYEE

EmpID → EmpName, DeptID, Salary

EmpID is the key, so the determinant is a superkey.

DEPARTMENT

DeptID → DeptName, ManagerID

DeptID is the key.

MANAGER

ManagerID → ManagerName

ManagerID is the key.

Thus, non-key attributes do not depend transitively on another non-key attribute within the same relation.

Therefore, the decomposition satisfies **3NF**.

6)

1. Given Relation

```
LIBRARY(
   MemberID,
   MemberName,
   BookID,
   Title,
   Author,
   Genre,
   BorrowDate
)
```

Where:

- MemberID – uniquely identifies a library member
- MemberName – name of the member
- BookID – uniquely identifies a book
- Title – title of the book
- Author – author of the book
- Genre – genre of the book
- BorrowDate – date on which the member borrowed the book

2. Functional Dependencies

The important functional dependencies are:

MemberID → MemberName

A member ID determines the member's name.

BookID → Title, Author, Genre

A book ID determines the book's title, author, and genre.

The borrowing information is identified by the combination:

(MemberID, BookID) → BorrowDate

This means a particular member borrowing a particular book determines the borrowing date.

3. Multi-Valued Dependency

A member can borrow **multiple books**.

Therefore, the relationship between a member and the set of books borrowed can be represented as:

MemberID →→ BookID

This is a **multi-valued dependency (MVD)**.

It indicates that for one MemberID, there can be multiple independent BookID values.

For example:

M01 → B01
M01 → B02
M01 → B03

A member can borrow several books.

4. Problem in the Original Relation

Suppose:

Member M01 = Arun

and Arun borrows:

B01 = DBMS
B02 = Operating Systems

The original relation may contain:

MemberID	MemberName	BookID	Title	Author	Genre	BorrowDate
M01	Arun	B01	DBMS	Korth	CS	01-08-2026
M01	Arun	B02	OS	Galvin	CS	05-08-2026

If member information and book information are repeatedly stored with every borrowing record, redundancy occurs.

This can cause:

Update Anomaly

If the name of a member changes, it may need to be updated in multiple rows.

Insertion Anomaly

A new book cannot be stored independently if the relation requires a borrowing record.

Deletion Anomaly

Deleting the only borrowing record for a book may accidentally remove information about that book.

5. Why 4NF is Required

A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial multivalued dependency:

X →→ Y

X must be a **superkey**.

In the original relation:

MemberID →→ BookID

But MemberID alone is not a superkey of the complete LIBRARY relation.

Therefore, the relation violates **4NF**.

6. Decomposition into 4NF

Separate the independent information into different relations.

Relation 1 – MEMBER

```
MEMBER(
   MemberID,
   MemberName
)
```

Primary Key:

MemberID

Functional dependency:

MemberID → MemberName

Relation 2 – BOOK

```
BOOK(
   BookID,
   Title,
   Author,
   Genre
)
```

Primary Key:

BookID

Functional dependency:

BookID → Title, Author, Genre

Relation 3 – BORROW

```
BORROW(
   MemberID,
   BookID,
   BorrowDate
)
```

Primary Key:

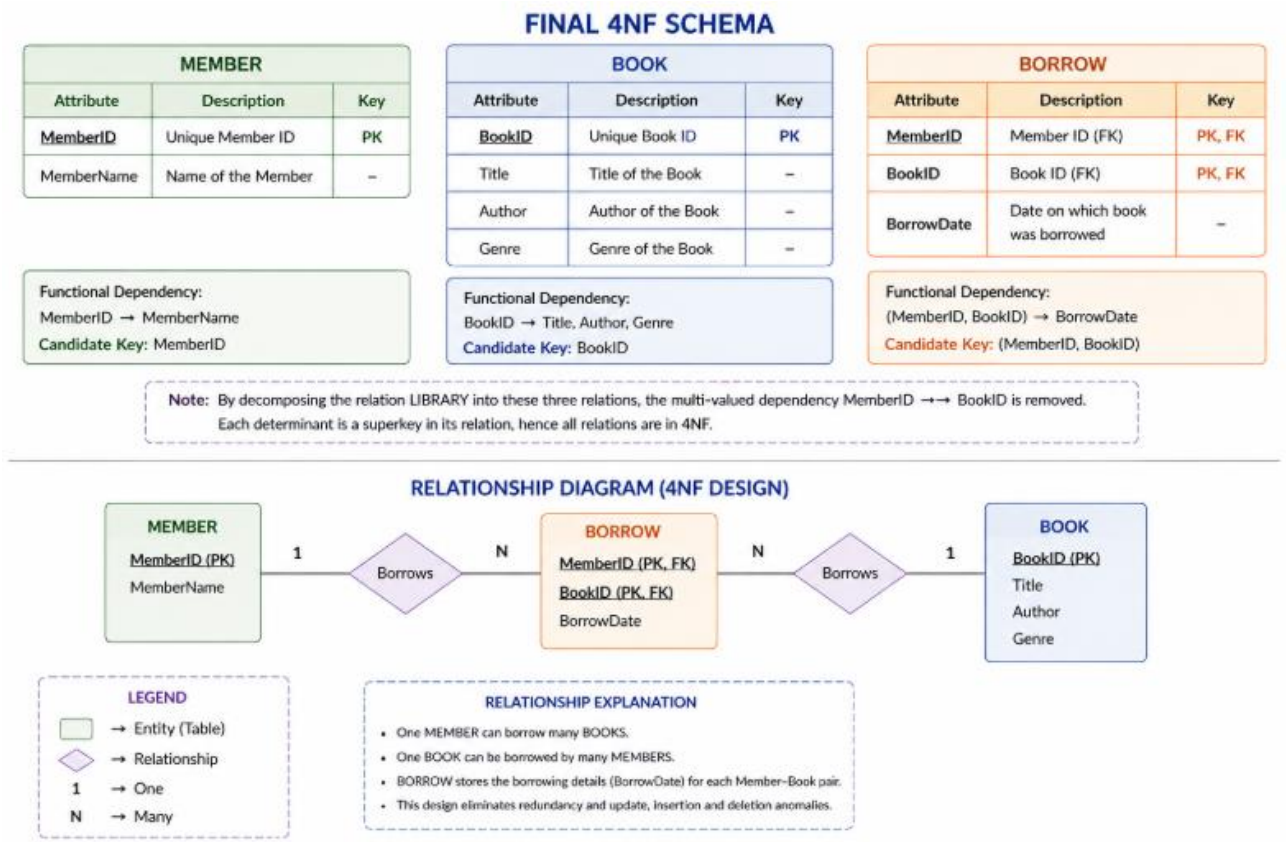
(MemberID, BookID)

Foreign Keys:

MemberID → MEMBER(MemberID)

BookID → BOOK(BookID)

7. Final 4NF Schema



7)

1. Given Relation

BILLING(
BillID,
CustomerID,
CustomerName,
PlanID,
PlanName,
CallMinutes,
Amount
)

2. Functional Dependencies

The important functional dependencies are:

BillID → CustomerID, PlanID, CallMinutes, Amount

CustomerID → CustomerName

PlanID → PlanName

Explanation

- BillID uniquely identifies one billing record.
- CustomerID identifies a particular customer, so it determines CustomerName.
- PlanID identifies a particular telecom plan, so it determines PlanName.
- CallMinutes and Amount are specific to the billing record identified by BillID.

3. Candidate Key

To find the candidate key, calculate the closure of BillID.

BillID+ = {BillID}

Using:

BillID → CustomerID, PlanID, CallMinutes, Amount

we get:

BillID+ =
{BillID, CustomerID, PlanID, CallMinutes, Amount}

Using:

CustomerID → CustomerName

we get:

BillID+ =
{BillID, CustomerID, CustomerName,
PlanID, CallMinutes, Amount}

Using:

PlanID → PlanName

we get:

BillID+ =
{BillID, CustomerID, CustomerName,
PlanID, PlanName, CallMinutes, Amount}

The closure contains **all attributes of the relation**.

Therefore:

Candidate Key = BillID

Since the candidate key is minimal, it can also be selected as the primary key.

Primary Key = BillID

4. Key Summary

Type	Attribute
Candidate Key	BillID
Primary Key	BillID
Foreign Key	CustomerID
Foreign Key	PlanID

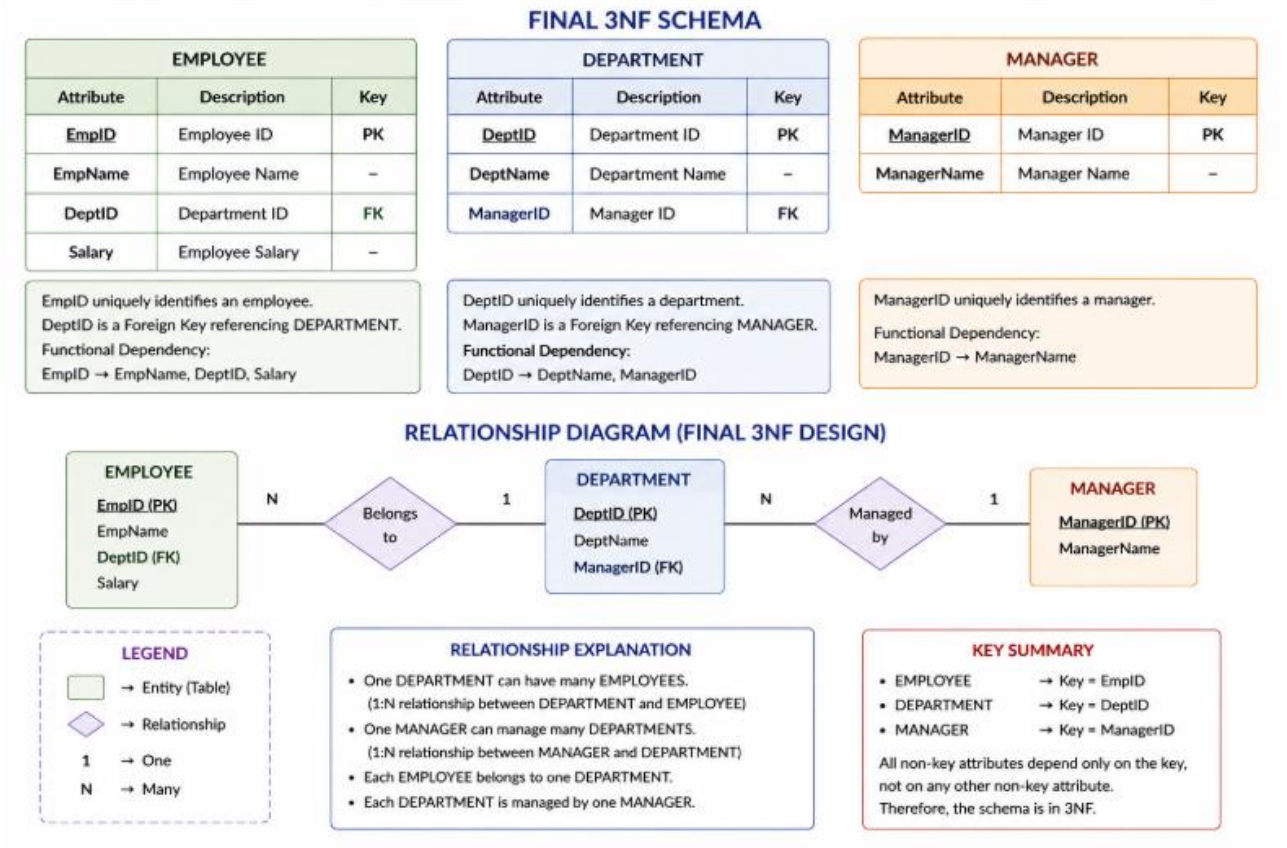
5. Dependency Summary

BillID → CustomerID, PlanID, CallMinutes, Amount

CustomerID → CustomerName

PlanID → PlanName

The dependencies show that customer and plan information can be separated during normalization to reduce redundancy and avoid update, insertion, and deletion anomalies.



8)

1. Functional Dependencies

Assuming each product has one supplier:

PID → PName, SupplierID, Category, Price, Warehouse

SupplierID → SupplierName

Here, PID uniquely identifies a product, while SupplierID identifies a supplier.

2. Candidate Key

Calculate the closure:

PID+ = {PID}

Using:

PID → PName, SupplierID, Category, Price, Warehouse

we get:

PID+ = {PID, PName, SupplierID, Category, Price, Warehouse}

Using:

SupplierID → SupplierName

we get:

PID+ = {PID, PName, SupplierID, SupplierName,
Category, Price, Warehouse}

Since all attributes are obtained:

Candidate Key = PID

3. 1NF

The relation is in **1NF** because all attributes contain atomic values and there are no repeating groups.

PRODUCT(
PID, PName, SupplierID, SupplierName,
Category, Price, Warehouse
)

4. 2NF

The candidate key is only PID, which is a **single attribute**.

Therefore, partial dependency is not possible.

Hence, the relation is already in:

2NF

5. 3NF

There is a transitive dependency:

PID → SupplierID
SupplierID → SupplierName

Therefore:

PID → SupplierName

SupplierName indirectly depends on PID.

So the relation is **not in 3NF**.

Decompose into:

PRODUCT

PRODUCT(
PID,
PName,

SupplierID,
Category,
Price,
Warehouse
)

SUPPLIER

SUPPLIER(
SupplierID,
SupplierName
)

6. BCNF Check

PRODUCT

PID → PName, SupplierID, Category, Price, Warehouse

PID is the key of PRODUCT.

Therefore, it satisfies BCNF.

SUPPLIER

SupplierID → SupplierName

SupplierID is the key of SUPPLIER.

Therefore, it also satisfies BCNF.

Hence, the final decomposition is in **BCNF**, which is stronger than 3NF.

9)

1. Initial Relation

A poorly designed relation can be represented as:

SCHOOL(
StudentID,
StudentName,
ClassID,
ClassName,
TeacherID,
TeacherName,
Grade
)

2. Functional Dependencies

StudentID → StudentName, ClassID, Grade

ClassID → ClassName, TeacherID

TeacherID → TeacherName

Explanation

- StudentID uniquely identifies a student and their details.
- ClassID identifies a class and its class teacher.
- TeacherID identifies a teacher.

The source scenario also identifies StudentID, ClassID, and TeacherID as primary keys in their respective relations, with ClassID and TeacherID used as foreign keys.

3. Candidate Key

Since StudentID determines all the student, class and teacher information through the dependencies:

StudentID+ =
{StudentID, StudentName, ClassID, Grade,
ClassName, TeacherID, TeacherName}

Therefore:

Candidate Key = StudentID

4. Problems in the Original Relation

The single table creates redundancy.

Update anomaly:
If a teacher's name changes, it may have to be updated in many student records.

Insertion anomaly:
A new teacher or class may not be stored properly until a student is associated with it.

Deletion anomaly:
Deleting the last student of a class could also remove the only stored information about that class or teacher.

5. BCNF Decomposition

STUDENT

STUDENT(
StudentID (PK),
StudentName,
ClassID (FK),
Grade
)

CLASS

CLASS(
ClassID (PK),
ClassName,
TeacherID (FK)
)

TEACHER

TEACHER(
TeacherID (PK),
TeacherName
)

10)

1. Initial Relation

MOVIE(
MovieID,
ActorID,
DirectorID,
GenreID
)

A movie can have:

- multiple actors,

- multiple directors,
- multiple genres.

These relationships are independent.

2. Join Dependencies

The relation contains the following join dependency:

*(MovieID, ActorID),
(MovieID, DirectorID),
(MovieID, GenreID)

This means the complete MOVIE relation can be reconstructed by joining the three smaller relations.

3. Decomposition

MOVIE_ACTOR

MOVIE_ACTOR(
MovieID,
ActorID
)

MOVIE_DIRECTOR

MOVIE_DIRECTOR(
MovieID,
DirectorID
)

MOVIE_GENRE

MOVIE_GENRE(
MovieID,
GenreID
)

4. 5NF Design

MOVIE_ACTOR

MovieID (PK, FK)
ActorID (PK)

MOVIE_DIRECTOR

MovieID (PK, FK)
DirectorID (PK)

MOVIE_GENRE

MovieID (PK, FK)
GenreID (PK)

5. Lossless-Join Verification

The original relation can be reconstructed as:

MOVIE
=
MOVIE_ACTOR
⋈ MOVIE_DIRECTOR

⋈ MOVIE_GENRE

The decomposition is **lossless** when joining these relations produces only valid movie combinations and does not introduce spurious tuples.

For example, if movie M01 has:

Actors: A1, A2
Directors: D1
Genres: G1, G2

the three relations independently store these associations.

Joining them using MovieID reconstructs the original movie relationships.

6. Why 5NF?

The decomposition removes independent many-to-many relationships from the single relation.

Each resulting relation represents only one relationship:

Movie ↔ Actor
Movie ↔ Director
Movie ↔ Genre

Therefore, the decomposition eliminates redundancy caused by join dependencies and gives a **5NF design**.